C11 Subscription vs API: Cost Patterns

Design Intermediate

Understanding when each billing model is advantageous

Two fundamentally different models

Subscription: you pay a fixed monthly amount and consume within a token budget limit. The cost is predictable, but the budget is capped and the limits are deliberately opaque.

API: per-token billing. Set and verify spending limits for unattended work; cost depends on all calls and retries.

The practical distinction: subscription favors solo developers with regular daily usage, API favors teams, CI/CD pipelines, and high-volume use cases.

How subscription limits work

Subscriptions use a hybrid model: a 5-hour rolling window and a weekly cap. Both apply simultaneously. The allowance is not published as a fixed token quantity; model, context and tools affect consumption.

Quota is not an API price ratio. Model, context, tools and account policy affect subscription consumption. Inspect `/usage` and compare real sessions; the API token prices below do not predict how quickly a weekly allowance runs out.

Strategies by plan

Plan	Recommendation
Pro	Choose an available model for the task; monitor `/usage`
Max 5x	Explicit Sonnet routing can be evaluated against the default
Max 20x	More Opus freedom, but monitor weekly consumption
API	Haiku for mechanical tasks, Sonnet for dev, Opus for audit

The OpusPlan pattern for saving

On a limited budget, compare Opus for planning with Sonnet or Haiku for bounded execution. Measure retries and accepted results before assuming this routing saves money.

```
# Phase 1: planning (Opus)
claude --model opus
"Analyze the architecture and propose a migration plan"

# Phase 2: execution (Sonnet/Haiku)
claude --model sonnet
"Implement the plan according to the defined specs"
```

`/model opusplan` specifically uses Opus for planning and Sonnet for execution; it does not select Haiku.

Consumption monitoring

```
/usage          # Costs, usage limits and statistics
/context        # Token breakdown

# Community tool for cross-session history
ccusage         # Overview all periods
ccusage daily   # Daily totals
ccusage monthly # Monthly totals
ccusage --breakdown # By model
```

`/usage` provides built-in metrics; `ccusage` adds local cross-session analysis. Neither estimate replaces an invoice.

When API wins

Subscription hits its limits in three cases: teams where multiple developers consume simultaneously, CI/CD pipelines with a high PR volume, and projects with long daily sessions that exhaust the budget before the end of the week. Compare total API cost and accepted results against the subscription allowance before switching.

Explicit token-budget examples

Standard direct-API USD, excluding caching and fast mode:

Input / output	Sonnet 5.5	Opus 5.5
50K / 5K	$0.15	$0.30
150K / 20K	$0.50	$1.00

Actual task cost depends on all calls and retries.